Self Without a Transcript

Reading, changing and reusing a conditional policy

November 4, 2026

Abstract

Can a model change a policy after discarding the words that specified it? On eight familiar topics, a probe of frozen Gemma states recovered the order of verdicts on 2,043/2,048 rephrased records, but the complete preference on only 1,311/2,048. We then trained a compiler to store two conditional actions, a small map to revise the trigger and a native Gemma reader to choose an action. With familiar clause patterns and new signal names, all 3,072 choices were correct. Replacing the midpoint with the opposed policy reversed every choice. Bypassing the update missed all 1,280 judgments whose correct action changed. New clause wording reduced correct choices to 1,140/1,536. Applying the same updater twice on the familiar wording gave 4,608/9,216. Thus a learned policy memory supported a single source-free revision within its trained format. Its own correct output could fall outside the format it knew how to revise.

1 A rule for choosing a route

A walker says, "I take the river path when signals Alder and Birch are both on. Otherwise I take the ridge path." Tomorrow Alder is on and Birch is off. The walker should take the ridge path. Now suppose the original sentence is gone. Could a model work from its current representation, expand the rule into cases, turn those cases back into a rule, and still choose the ridge path when asked tomorrow? What would change if it had to put its intermediate state into words?

Form of the same policy	What the model would need to preserve
Original description	Take the river path when both signals are on. In every other situation take the ridge path.
Exhaustive cases	Off/off, ridge. Off/on, ridge. On/off, ridge. On/on, river.
Reintegrated rule	Take the ridge path by default. Switch to the river path exactly when both signals are on.

The opposed policy exchanges river and ridge in every row. Exchanging the order of sentences changes neither policy. These transformations make the research question testable, but they also create an ambiguity. A model that simply copies the initial policy can give the correct final answer. Evidence for a useful update must therefore identify what the intermediate state supports, whether replacing it changes the later choice, and what is lost when an update is omitted.

We first ask what a question-free state preserves about a preference, then whether exchanging a state changes a native answer. We use two flags to give the walker a conditional policy. Passing a conjunction bit between two calls still lets the second call consult the original policy. Our final experiment removes those words and tests both a single revision and the reuse of its output. The first-person frame supplies the policy. Establishing an effect specific to self-attribution would require matched third-person and impersonal descriptions.

2 What is read, and by whom?

Reading a state. An encoder reads a description before any evaluation question appears. We retain eight vectors at a specified layer and call this state $E(s)$. A source probe learns a score from these vectors to predict a feature of the description. A high probe score establishes that the fitted reader can recover that feature from one state. In a separate test, we inject a state into a task prompt and read the language model's own next token. We call this the *native answer*. An *opposed donor* supplies the state of the other policy while the question and its displayed options stay fixed.

Reading a choice. Every binary action question is presented twice, with the options exchanged. Native correctness requires the most likely token over the full vocabulary to be the requested letter. Other tokens count as errors. A strict unit in the conditional experiments fixes topic, flag-name pair, wording and representation stage. It passes only when both policies are correct in all four worlds, in every genuine source order and in both label orders. For example, a reader that always follows the last clause can score many individual decisions and still fail the entire unit. The original description and the table each have two orders. The reintegrated rule has one realization per wording family.

Reading across updates. The eventual comparison has three paths. *Rewrite in words* (`T`) passes generated text to the next step. *Continue in slots* (`L`) passes the updated hidden state. *Say it, encode it, then continue* (`B`) verbalizes that same intermediate state and encodes the words before the next update. The encoder, verbalizer and updater have internal identifiers E, D and G. The learned operation sequence is expansion into cases (G_1) followed by reintegration into a rule (G_2). The primary language intervention shares the same first-stage state across the last two paths. Generation proceeds to native end of sequence when context permits, with the actual stopping reason and any censored text retained.

Earlier Mistral-7B-Instruct-v0.3 and Gemma-2-2B-IT probes explain the ordering of our tests. Both models answered 24/24 preference questions from literal descriptions. Twenty-four distinct updated slot tensors nevertheless produced one verbalization per checkpoint, and all 12 re-encoded donor contrasts tied. Replaying states from positions that had already processed both options could reproduce native lexical choices. A recurrence needs a state formed *before* the question. The original tests of uncalibrated transport, output positions and stopping conditions are indexed in the supplement. Development experiments 1–8

Related work

The Quine Test studies whether descriptions reproduce preference profiles in fresh instances [1]. Propositional probes distinguish facts and their relational bindings in context [2]. Our verdict probe asks which of two familiar judgment roles survives a change in the action words. The donor and branch exchanges address the separate question of causal use. Coconut trains reasoning through continuous hidden-state feedback [6], and recurrent-depth models allocate repeated computation within a network [5]. Natural Language Autoencoders provide a learned activation-to-language-to-activation interface [3]. The planned language insertion compares later use on the two paths from the very same intermediate state.

3 A frozen state preserves the order of familiar verdicts

Consider a two-record description that accepts the river route and rejects the ridge route. Hold the two actions and their mention order fixed, then exchange the verdicts. The two descriptions

contain the same tokens and the same slot positions, yet which action is accepted changes. A reader using only one question-free state can be tested on each member independently.

We froze Gemma-2-2B-IT and trained a linear score with a fitted offset on 4,608 such records. The readout predicted whether the first record accepted its action. A second readout predicted which action was mentioned first within an inferred topic. Combining the two predictions selected the preferred action. Evaluation used 64 unfamiliar action expressions across eight known topics, two familiar verdict vocabularies, both field placements, both action orders and both policies. The 2,048 states are repeated realizations of 16 policies within those topics.

Question asked of one held-out state	Correct	Comparison or consequence
Is the first action accepted?	2,043/2,048	Shuffling the training role supervision gave 1,022/2,048.
Are both members of an opposed pair read correctly?	1,019/1,024 pairs	Each member was read separately.
Which specific action is preferred?	1,311/2,048	Topic routing and action identity also have to be correct.

The role result passed its registered criterion in all four combinations of verdict vocabulary and field placement. A fit without extra weight on opposed pairs was equally accurate on the role. Each description contains one acceptance and one rejection, so a cue in either record can reveal their order. The result establishes a recoverable *order of familiar verdicts* under action rephrasing. The complete-policy errors locate the next difficulty. In 527 of the 737 errors, topic and verdict order were correct but the reader identified the wrong first action. Another 206 involved an incorrect topic and four identified the right action but assigned it the wrong verdict within the correct topic. Probe, shuffled control and error partition

4 When a source state changes a model answer

A separate intervention asks whether a source can drive a native choice. We first established that eight vectors derived from explicit option words could influence a frozen Gemma reader. We then fitted a map from a question-free source encoding into this working input space. At test time the map received one source state. The downstream question contained the displayed alternatives, and the model produced its ordinary next-token scores. Source and recipient were frozen during fitting of the map. Word-derived input control

On the 32 fitting states the mapped sources gave 64/64 native choices. Opposed donors reduced the score against the original gold to 0/64 and reversed the A/B margin on all 64 judgments. The fitting mean and zero inputs each gave 32/64. Wording of the question transferred better than wording of the source. The same states gave 62/64 under a withheld question wording, whereas changed source wording yielded 57/128 and two new topics yielded 30/64. This establishes causal use of the supplied preference on its fitting bank. General conditional reading requires a further operation. The same stored policy now has to yield *different* actions depending on the later world. Source exchange and transfer panels

We tested that operation on the eight known action topics by supplying each original description, case table and reintegrated rule independently. These were experimenter-authored teachers. None was generated by a learned update. Native choices from ordinary slots were 127/256, 128/256 and 64/128, respectively, against balanced zero-state counts of 128/256, 128/256 and 64/128. Literal text gave 219/256, 215/256 and 88/128. Because even the original text left errors, the next experiment trained a reader on text before admitting slot learning. Independent teacher controls

Literal training and one fixed continuation raised fitting accuracy to 3,751/3,840 (97.7%). The corresponding strict counts were 23/48 for the original description, 28/48 for the table and 47/48 for the rule. Only the rule cleared the fitting requirement of 95% native and 90% strict. New names, source wording and questions gave 923/1,280 native choices. Of 89 fitting errors, 85 returned B when A was correct. This discrepancy motivates the curriculum comparison. It keeps both label orders, separates changes in names, grammar and questions, and asks where the decision fails when its parts can be tested independently. Literal curriculum · Margin continuation and label-order diagnostic

5 From two flags to an action

We compared two training curricula at equal update, target and padded-token budgets. Both began from the same saved literal reader on a frozen Gemma backbone. *Practise the final choice* (`direct`) spends every target on choosing the action from the policy and the two current flag values. *Learn the parts of the choice* (`parts`) splits its targets equally among reading one flag, deciding whether both are on, selecting an action from a supplied branch value, and answering the ordinary action question. Both arms train the same rank-16 query/value adapters in the final four layers. They use the same ordered source units, loss and fresh optimizer settings for 384 updates of 32 answers. Alongside full-vocabulary cross entropy, both include a gold-letter margin and a penalty for a common A/B bias under a swap of the displayed options.

We fixed the final weights before scoring any new panel. Familiar or new indicator names, source grammar and question wording form eight crossed cells. Each cell includes 1,280 native decisions on 160 descriptions. The primary comparison uses the cell in which all three factors changed. A training benefit requires a net gain of at least 64 decisions (five percentage points), gains in at least five of eight topics, and no loss in the total number of strict units. This paired test concerns the allocation of a fixed training budget. The historical absolute thresholds for using the literal reader are reported separately. Prospective design and decision rules

The secondary endpoint tests an explicit intermediate state. The first call receives only the two readings and answers "Are both on?" The answer's displayed label maps to a single Boolean value. The second call receives the policy and a fixed verbal rendering of that bit. It has no access to the individual readings. We also complement the bit at fixed policy or exchange the policy at fixed bit. Both interventions should reverse the final action. The two calls have independently swapped answer labels. Identical calls are reused, and an invalid first answer counts against every path that needs it. A separate true-bit oracle localizes errors after the world test. This experiment uses a hand-specified one-bit message and retains the original policy in the second call. For the walker, a false bit should lead to the ridge and exchanging it for true should send the walker to the river. A learned hidden-state updater would also have to carry the policy itself.

Both arms received 12,288 labels, 3,072 training forwards and 1,747,152 padded training tokens. The number of unmasked tokens was 1,747,152 for final-choice training and 1,369,704 for component training. On the jointly new panel the component arm recovered 19 answers that the final-choice arm missed and lost 25 that it got right. The net was $-6/1{,}280$ with gains in four of eight topics. The primary benefit criterion failed. Changing names alone left the final-choice arm at 1,276/1,280, its familiar count. Changing source grammar alone reduced it to 1,209/1,280. The component arm showed a similar pattern, scoring 1,272 and 1,199 respectively. These contrasts concern the specified templates.

Jointly new stage	Final choice, native	Parts, native	Final choice, strict	Parts, strict
Description	460/512	459/512	1/16	1/16
Exhaustive cases	470/512	468/512	3/16	0/16
Reintegrated rule	235/256	232/256	6/16	3/16
All three	1,165/1,280	1,159/1,280	10/48	4/48

For the one-bit route, both arms read every individual flag correctly (32/32). The final-choice arm recognized the conjunction on 13/16 native questions, the component arm on 15/16. In the latter arm, when Nacre was on, Flint was off and A meant yes, the model answered A to the question of whether both were on. It correctly answered the same question with the labels reversed. That one error failed the per-order criterion. The second call can reuse the same bit across many policies, so 2,560 composed paths draw on just 16 conjunction judgments and 640 distinct second-call responses per arm.

Training	Predicted bit	True bit	Both bit settings correct	Strict units
Final choice	1,932/2,560	2,328/2,560	1,718/2,560	0/48
Learn parts	2,234/2,560	2,370/2,560	2,049/2,560	11/48

Component supervision improved the observed two-call score by 302 paths (11.8 percentage points) relative to the matched training control. Supplying the true bit increased this arm's count by a further 136, leaving action errors even with the world test solved. The 11 strict units comprised 3/16 descriptions, 2/16 tables and 6/16 rules. Correct original and opposed-policy answers occurred on 2,072/2,560 paths. The predeclared constructive criterion required at least 90% final correctness, 75% strict units and 90% correct paired reversals for both interventions at *each* stage, as well as 90% on each component under each label mapping. It failed. The observed two-call gain is a descriptive comparison under one schedule and one checkpoint. Its 87.3% accuracy also falls below the same arm's ordinary 90.5% accuracy. All eight panels, stage counts and native intervention records

6 Changing the policy after its words are gone

The walker takes the ridge by default and the river when both lamps are on. Now change the trigger to either lamp. With just one on, the walker should switch from ridge to river. With both on, the route should stay the same. We use these changed and unchanged worlds to test whether a later computation actually revises a remembered policy.

A frozen Gemma encoded the two clauses separately, before seeing the question, world or displayed alternatives. Two supervised heads predicted each clause's role and action over a fixed vocabulary of 16 actions. Combining their soft probabilities gave a distribution for each branch. We supplied the initial four-world layout, which uses the default distribution three times and the joint-trigger distribution once. A separately trained four-by-four map for each requested trigger (both, either, or exactly one lamp on) revised the whole table. Only then did an external world address select a row. A native Gemma reader received that continuous mixture of action embeddings and the displayed alternatives. No token decoding, source text or source attention crossed the boundary. The compiler, update and reader were frozen before evaluation.

Both panels used new lamp names, both clause orders and both A/B orders. The primary panel used two familiar clause patterns. The challenge used a new contrast between reaching full

illumination and falling short of it. A correct opposed exchange required the original and donor choices to be correct with a reversed A/B margin. Each strict unit required all 96 decisions across polarities, orders, worlds and operations for a topic, lamp pair and wording.

Outcome	Familiar wording	New wording
Correct complete midpoint	128/128	32/64
Native action after one edit	3,072/3,072	1,140/1,536
Correct original and opposed reversal	3,072/3,072	744/1,536
All 96 judgments in a strict unit	32/32	3/16

On familiar wording, exchanging only the switched-action row changed the correct choice on all 1,536 affected judgments and preserved it on all 1,536 unaffected judgments. Replacing only the default rows gave the same counts. Updating solved all 1,280 judgments whose correct choice changed, while skipping the update solved zero. On the other 1,792, both paths were completely correct. Uniform memory gave 1,536/3,072. All registered primary criteria passed. These are repeated decisions over 16 policies on eight known topics.

On new wording only 32 of 64 midpoints recovered both branch actions. In 20 incomplete memories both branches named the same action on the correct menu. In another 12 at least one branch pointed outside the menu. The correct one-hot policy drove all 1,536 answers through the same update and reader. The wording challenge missed its gate. Across both panels and eight interventions, 36,864 requests shared 1,740 distinct native inputs. Native outputs and intervention scores

We then froze a follow-up test on the already scored primary panel. A second edit replaced the first trigger while preserving the two branch actions. The updater received its own complete output and the new operation, without the first operation's name. When the first trigger was both-on, the second edit was correct on 3,072/3,072 judgments across the three requested triggers. After either-on the count was 1,536/3,072. After exactly-one-on it was 0/3,072. Opposed donors still flipped all 3,072 answers in that last condition, but both answers in each pair contradicted their policies. Only three of nine sequences met the reuse criterion. All first edits on these policies had been correct. Frozen follow-up design

The incoming layout explains the reversal. During fitting the first three rows always carried the default action. The learned map assigned these positions equal weights when reading the default. After an exactly-one edit, two of the three carried the switched action, and the fourth row now held the default. The same read operation then exchanged the two roles. An informative state can therefore be an unusable input for the particular updater trained to consume it.

7 From one revision to learning across revisions

The constructed memory establishes source-free conditional use on a fixed action vocabulary. It also exposes two distinct requirements for recurrence. A new source wording must bind the correct actions, and the updater must preserve a format that it can consume again. The single-bit reader in Section 5 had retained the literal policy. Here the next computation used the numeric table alone. The table format, world address and action vocabulary were supplied by design.

Our next test would train on whole sequences of revisions with exceptions, redundant edits and rule reintegration. Rollouts must include the learner's own intermediate errors. Branch a saved midpoint into an edit, a bypass and a donor exchange, and score them on independent changed and unchanged worlds. A reward for improvement alone can favor a poor first answer. Reward

correctness before and after the edit, audit a separately held verifier, and measure loss of previously correct choices. These comparisons connect the observed failure of reuse with work on multi-turn self-correction and reward overoptimization [7, 4].

For the original eight-slot hypothesis, test independently encoded descriptions, cases and reintegrated rules before comparing direct continuation with a language insertion from the same midpoint. For a scaling claim, compare recurrent reinforcement learning with direct-answer and supervised recurrent baselines at equal training and inference compute, including verifier calls and verbalization. Vary depth, task diversity and model size separately. Hold out whole operation and action families and measure transfer to independent planning and language tasks, retention, and variability across seeds. General capability and a contribution specific to self-reference require those additional comparisons. Protocols and scaling comparisons

Supporting material

The development record and experiment key names each earlier experiment and preserves its run identifiers, results tables and links to the raw records. The reproduction instructions give the execution seals. The reference audit records source and version information.

References

[1] Bibin Babu, Ivan P. Yamshchikov, and Aleksei Shpilman. The quine test: How much of a model's self survives its own description?, October 2026. URL https://arxiv.org/abs/2610.38726v1. Version 1 submitted 28 October 2026; preprint.

[2] Jiahai Feng, Stuart Russell, and Jacob Steinhardt. Monitoring latent world states in language models with propositional probes, 2024. URL https://arxiv.org/abs/2406.19501v2. Version 2 revised 6 December 2024; first submitted 27 June 2024.

[3] Kit Fraser-Taliente, Subhash Kantamneni, Euan Ong, Dan Mossing, Christina Lu, Paul C. Bogdan, Emmanuel Ameisen, James Chen, Dzmitry Kishylau, Adam Pearce, Julius Tarng, Alex Wu, Jeff Wu, Yang Zhang, Daniel M. Ziegler, Evan Hubinger, Joshua Batson, Jack Lindsey, Samuel Zimmerman, and Samuel Marks. Natural language autoencoders produce unsupervised explanations of llm activations. *Transformer Circuits Thread*, 2026. URL https://transformer-circuits.pub/2026/nla/index.html.

[4] Leo Gao, John Schulman, and Jacob Hilton. Scaling laws for reward model overoptimization, 2022. URL https://arxiv.org/abs/2210.10760.

[5] Jonas Geiping, Sean McLeish, Neel Jain, John Kirchenbauer, Siddharth Singh, Brian R. Bartoldson, Bhavya Kailkhura, Abhinav Bhatele, and Tom Goldstein. Scaling up test-time compute with latent reasoning: A recurrent depth approach, 2025. URL https://arxiv.org/abs/2502.05171.

[6] Shibo Hao, Sainbayar Sukhbaatar, DiJia Su, Xian Li, Zhiting Hu, Jason Weston, and Yuandong Tian. Training large language models to reason in a continuous latent space, 2024. URL https://arxiv.org/abs/2412.06769.

[7] Aviral Kumar, Vincent Zhuang, Rishabh Agarwal, Yi Su, John D. Co-Reyes, Avi Singh, Kate Baumli, Shariq Iqbal, Colton Bishop, Rebecca Roelofs, Lei M. Zhang, Kay McKinney, Disha Shrivastava, Cosmin Paduraru, George Tucker, Doina Precup, Feryal Behbahani, and Aleksandra Faust. Training language models to self-correct via reinforcement learning, 2024. URL https://arxiv.org/abs/2409.12917v2.